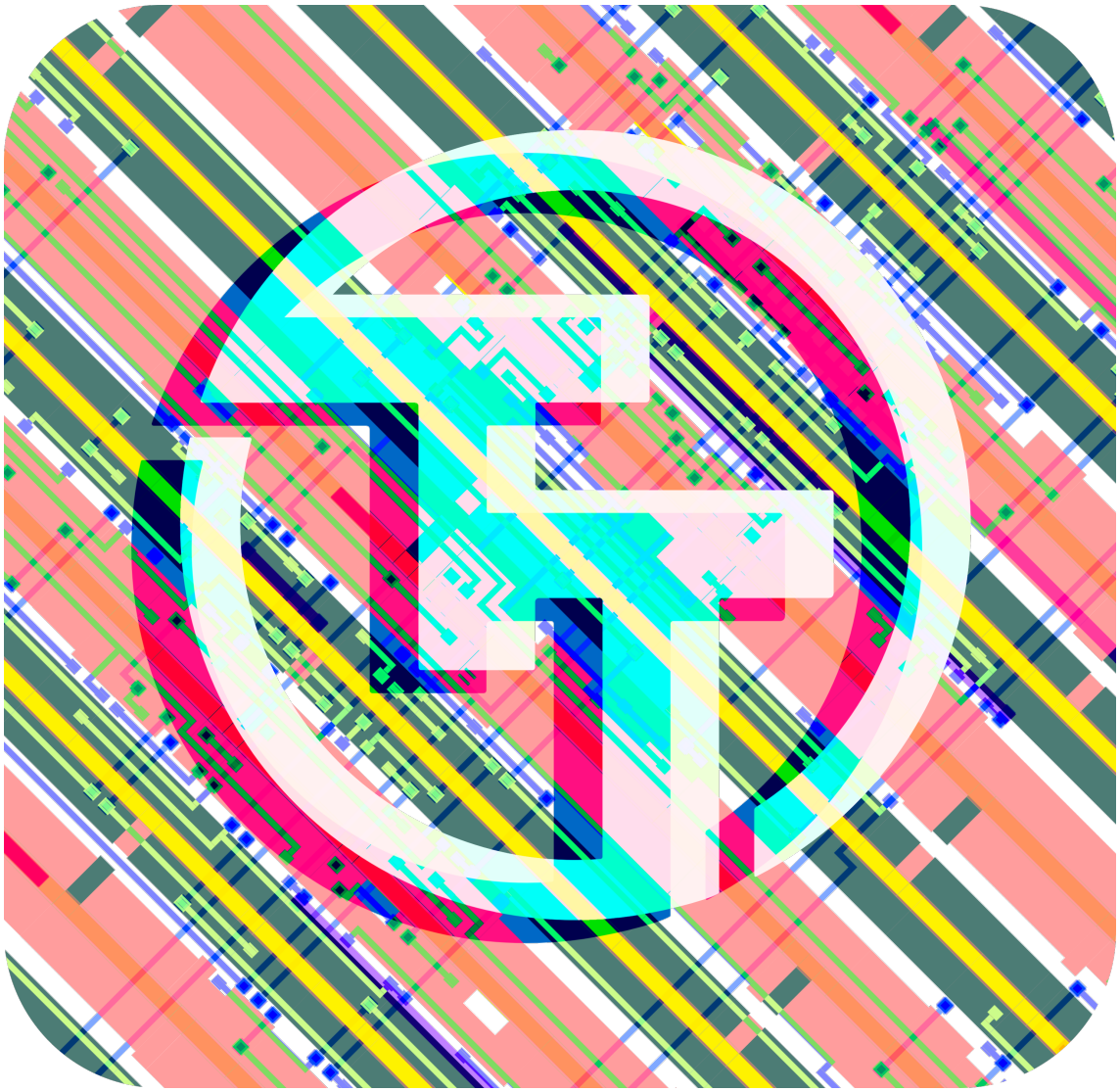


Tiny Tapeout SKY 25b Datasheet

Project Repository

https://github.com/chipfoundry/nydesign-cc2509.git

December 6, 2025

Contents

Chip map

Full chip map

GDS render

Logic density (local interconnect layer)

Projects

Chip ROM [0]

- Author: Uri Shaked
- Description: ROM with information about the chip
- GitHub repository
- HDL project
- Mux address: 0
- Extra docs
- Clock: 0 Hz

How it works

ROM memory that contains information about the Tiny Tapeout chip. The ROM is 8-bit wide and 256 bytes long.

The ROM layout The ROM layout is as follows:

Address	Length	Encoding	Description
0	8	7-segment	Shuttle name (e.g. “tt07”), null-padded
8	8	7-segment	Git commit hash
32	96	ASCII	Chip descriptor (see below)
248	4	binary	Magic value: `"TT\xFA\xBB"`
252	4	binary	CRC32 of the ROM contents, little-endian

The chip descriptor The chip descriptor is a simple null-terminated string that describes the chip. Each line is a key-value pair, separated by an equals sign. It contains the following keys:

Key	Description	Example value
shuttle	The identifier of the shuttle	tt07
repo	The name of the repository	TinyTapeout/tinytapeout-07
commit	The commit hash *	a1b2c3d4

- The commit hash is only included for Tiny Tapeout 5 and later.

Here is a complete example of a chip descriptor:

```
shuttle=tt07
repo=TinyTapeout/tinytapeout-07
commit=a1b2c3d4
```

How the ROM is generated The ROM is automatically generated by tt-support-tools while building the final GDS file of the chip. Look at the `rom.py` file in the repository for more details.

Reading the ROM There are two ways to address ROM, depending on the value of the `rst_n` pin:

1. When `rst_n` is high: Set the `ui_in` pins to the desired address.
2. When `rst_n` is low: Toggle the `clk` pin to read the ROM contents sequentially, starting from address 0.

In both cases, the ROM data for the selected address will be available on the `uo_out` pins, one byte at a time.

How to test

The first 16 bytes of the ROM are 7-segment encoded and contain the shuttle name and commit hash. You can dump them by holding `rst_n` low and toggling the `clk` pin, and observing the on-board 7-segment display.

Alternatively, you can keep `rst_n` high and set the `ui_in` pins to the desired address using the first four on-board DIP switches, while observing the on-board 7-segment display.

Pinout

#	Input	Output	Bidirectional
0	addr[0]	data[0]	
1	addr[1]	data[1]	
2	addr[2]	data[2]	
3	addr[3]	data[3]	
4	addr[4]	data[4]	

#	Input	Output	Bidirectional
5	addr[5]	data[5]	
6	addr[6]	data[6]	
7	addr[7]	data[7]	

Tiny Tapeout Factory Test [1]

- Author: Tiny Tapeout
- Description: Factory test module
- GitHub repository
- HDL project
- Mux address: 1
- Extra docs
- Clock: 0 Hz

How it works

The factory test module is a simple module that can be used to test all the I/O pins of the ASIC.

It has three modes of operation:

1. Mirroring the input pins to the output pins (when `rst_n` is low).
2. Mirroring the bidirectional pins to the output pins (when `rst_n` is high `sel` is low).
3. Outputing a counter on the output pins and the bidirectional pins (when `rst_n` is high and `sel` is high).

The following table summarizes the modes:

`rst_n`	`sel`	Mode	uo_out value	uio pins
0	X	Input mirror	ui_in	High-Z
1	0	Bidirectional mirror	uio_in	High-Z
1	1	Counter	counter	counter

The counter is an 8-bit counter that increments on every clock cycle, and resets when `rst_n` is low.

How to test

1. Set `rst_n` low and observe that the input pins (`ui_in`) are output on the output pins (`uo_out`).
2. Set `rst_n` high and `sel` low and observe that the bidirectional pins (`uio_in`) are output on the output pins (`uo_out`).

3. Set `sel` high and observe that the counter is output on both the output pins (`uo_out`) and the bidirectional pins (`uio`).

Pinout

#	Input	Output	Bidirectional
0	sel / in_a[0]	output[0] / counter[0]	in_b[0] / counter[0]
1	in_a[1]	output[1] / counter[1]	in_b[1] / counter[1]
2	in_a[2]	output[2] / counter[2]	in_b[2] / counter[2]
3	in_a[3]	output[3] / counter[3]	in_b[3] / counter[3]
4	in_a[4]	output[4] / counter[4]	in_b[4] / counter[4]
5	in_a[5]	output[5] / counter[5]	in_b[5] / counter[5]
6	in_a[6]	output[6] / counter[6]	in_b[6] / counter[6]
7	in_a[7]	output[7] / counter[7]	in_b[7] / counter[7]

TTSKY25A Register File Test [526]

- Author: Sylvain Munaut
- Description: Test structures for a future register file macro
- GitHub repository
- HDL project
- Mux address: 526
- Extra docs
- Clock: 0 Hz

How it works

It's basically a SRAM with 1 write port and 2 read ports. See design data for information

How to test

No test procedure has been written yet and it's not meant to be tested by the "demo board" but instead on a dedicated test platform.

External hardware

To do any meaningful timing testing you'll need some FPGA hardware to drive the various control signal in sequence with precise timings.

The exact testing platform is still TBD.

Pinout

#	Input	Output	Bidirectional
0	tbd	tbd	tbd
1	tbd	tbd	tbd
2	tbd	tbd	tbd
3	tbd	tbd	tbd
4	tbd	tbd	tbd
5	tbd	tbd	tbd
6	tbd	tbd	tbd

#	Input	Output	Bidirectional
7	tbd	tbd	tbd

Logic gate comparison [896]

- Author: Charles Brand
- Description: Compares two logic gates
- GitHub repository
- Wokwi project
- Mux address: 896
- Extra docs
- Clock: 0 Hz

How it works

Compares two logic gates output to confirm identical

How to test

Turn off and on any combination of inputs A-D

External hardware

None

Pinout

#	Input	Output	Bidirectional
0	A	E	
1	B	F	
2	C		
3	D		
4			
5			
6			
7			

Poly Workshop AND Gate2.1 [897]

- Author: Noah Marinuci
- Description: Two AND Gates to Turn two LEDs OFF
- GitHub repository
- Wokwi project
- Mux address: 897
- Extra docs
- Clock: 0 Hz

How it works

AND Gate turns LED off.

How to test

Turn each AND gate on and Off to test both LEDs.

External hardware

2 LEDs, Two AND gates.

Pinout

#	Input	Output	Bidirectional
0		A	
1	A	B	
2	B		
3	C		
4	D		
5			
6			
7			

3-floor-elevator-light [898]

- Author: Aaron Wang
- Description: lights up certain light given what floor the ‘elevator’ is on
- GitHub repository
- Wokwi project
- Mux address: 898
- Extra docs
- Clock: 0 Hz

How it works

This design uses flipflops to take one input of true or false, and gives different outputs depending on the current state. The state is updated manually with a push button to control the clock

How to test

have the input true, use button to move all the way “up” have the input false, use button to move all the way “down” be on the 2nd floor going “up” and then go “down” be on the 2nd floor going “down” and then go “up”

External hardware

3 LEDs, each on indicating a different floor 1 push button to control clock “up” or “down” switch

Pinout

#	Input	Output	Bidirectional
0	up or down on elevator		3rd floor light
1			2nd floor light
2			1st floor light
3			
4			
5			
6			

#	Input	Output	Bidirectional
7			

AND-OR Gates [899]

- Author: Michael Asiamah
- Description: Takes input from two AND gates, then passes them through an OR gate
- GitHub repository
- Wokwi project
- Mux address: 899
- Extra docs
- Clock: 0 Hz

How it works

Takes input from two AND gates, then passes them into an OR gate

How to test

Inputs A AND B can be flipped on, or inputs A AND C can be flipped on. No other combinations should yeild and output.

External hardware

List external hardware used in your project (e.g. PMOD, LED display, etc), if any

Pinout

#	Input	Output	Bidirectional
0	A	(A*B)+(*AC)	
1	B		
2	C		
3			
4			
5			
6			
7			

Open Source IC Design Comparator [900]

- Author: Theresa O'Connor
- Description: Basic two-bit comparator using 4 one-bit inputs
- GitHub repository
- Wokwi project
- Mux address: 900
- Extra docs
- Clock: 0 Hz

How it works

Two-bit comparator with bit0+bit1 being compared to bit2+bit3 (Ex. "00" compared to "10" should cause LED1 to be on).

How to test

Test combinations of the four inputs following the truth table where LED1 will be on if the first two bits are less than the last two bits, LED2 will be on if the first two bits are equal to the last two bits, and LED3 will be on if the first two bits are greater than the last two bits.

External hardware

3 LED Display.

Pinout

#	Input	Output	Bidirectional
0	bit0	LED1	
1	bit1	LED2	
2	bit2	LED3	
3	bit3		
4			
5			
6			
7			

#	Input	Output	Bidirectional

Poly Workshop FULL ADDER [901]

- Author: Jayden
- Description: FULL ADDER Gate Template
- GitHub repository
- Wokwi project
- Mux address: 901
- Extra docs
- Clock: 0 Hz

How it works

It's a full adder that adds two bits together.

How to test

Use the switches.

External hardware

Two LEDs.

Pinout

#	Input	Output	Bidirectional
0	A	SUM_moastsignificantbit	
1	B	SUM_leastsignificantbit	
2			
3			
4			
5			
6			
7			

Traffic Light IC Chip [902]

- Author: Alyssa Okosun
- Description: This is a traffic light that changes color every 13 secs.
- GitHub repository
- Wokwi project
- Mux address: 902
- Extra docs
- Clock: 10000 Hz

How it works

My project uses a 10kHz signal, which is slowed down by D-Flip Flops I hold the current state in D-Flip FLops and use logic gates to determine the next state.

How to test

I connected 3 LEDs to the output to test if the lit up at the right time, and in te right order.

External hardware

I used 3 LEDs to test if the lights lit up at the right timw, and in the right order.

Pinout

#	Input	Output	Bidirectional
0		RED	
1			
2			
3			
4		YELLOW	
5			
6			
7		GREEN	

BCD [903]

- Author: Nicholas Grieco
- Description: Binary Coded Decimal
- GitHub repository
- Wokwi project
- Mux address: 903
- Extra docs
- Clock: 0 Hz

How it works

It takes 3 inputs that are binary values and makes it into a decimal value.

How to test

BRuh use your project

External hardware

List external hardware used in your project (e.g. PMOD, LED display, etc), if any

Pinout

#	Input	Output	Bidirectional
0	A	Out	
1	B		
2	C		
3			
4			
5			
6			
7			

Demo Circuit [904]

- Author: Anindita
- Description: Full Adder, Comparator
- GitHub repository
- Wokwi project
- Mux address: 904
- Extra docs
- Clock: 0 Hz

How it works

Full Adder : In0, In1, In2 are inputs to the full adder (A, B and C) Out0 and Out1 are outputs (SFA and CFA) SFA = C XOR (A XOR B) CFA = (A OR B) AND (C OR (A XOR B)) AB=

1 bit comparator In3and In4. Out2, out3 and out 4. (lesser, equal and greater)

Half Adder In5 and In6 (A, B), Out 5 and out 6 (HAS and HAC)

How to test

Connect inputs signals and add LED's to check output

External hardware

LED display and resistors

Pinout

#	Input	Output	Bidirectional
0	A	SFA	
1	B	CFA	
2	C	Out1	
3	In1	Out2	
4	In2	Out3	
5	X	SHA	

#	Input	Output	Bidirectional
6 7	Y	CHA	

Poly Workshop 4-Bit-ADDER [905]

- Author: Noah Breedy
- Description: Example 4-Bit adder
- GitHub repository
- Wokwi project
- Mux address: 905
- Extra docs
- Clock: 0 Hz

How it works

A simple 4-Bit Adder

How to test

5 LEDS to show the output of the addition

External hardware

5 LEDS

Pinout

#	Input	Output	Bidirectional
0	a0	c0	
1	a1	c1	
2	a2	c2	
3	a3	c3	
4	b0	c4	
5	b1		
6	b2		
7	b3		

NSF_HVCC_2025 [906]

- Author: Aiden Fedorowicz
- Description: And gate circuit
- GitHub repository
- Wokwi project
- Mux address: 906
- Extra docs
- Clock: 0 Hz

How it works

A single and gate

How to test

Try all combinations of A and B

External hardware

A single LED

Pinout

#	Input	Output	Bidirectional
0	A	A AND B	
1	B		
2			
3			
4			
5			
6			
7			

Example AND Gate [907]

- Author: Jeffery Perez
- Description: Example AND Gate
- GitHub repository
- Wokwi project
- Mux address: 907
- Extra docs
- Clock: 0 Hz

How it works

A single AND gate

How to test

A single Led can be connected to the output

A single USB

LED

Pinout

#	Input	Output	Bidirectional
0	A	A AND B	
1	B		
2			
3			
4			
5			
6			
7			

Fei-Tian-AND-Gate [908]

- Author: Renata
- Description: AND gate
- GitHub repository
- Wokwi project
- Mux address: 908
- Extra docs
- Clock: 0 Hz

Pinout

#	Input	Output	Bidirectional
0	A	A AND B	
1	B		
2			
3			
4			
5			
6			
7			

Poly Workshop AND Gate [909]

- Author: Ariel Cruz
- Description: Suny Poly Workship AND Gate
- GitHub repository
- Wokwi project
- Mux address: 909
- Extra docs
- Clock: 0 Hz

How it works

A Single AND gate

How to test

A single LED can be connected to the output.

External hardware

A single LED

Pinout

#	Input	Output	Bidirectional
0	A	A AND B	
1	B		
2			
3			
4			
5			
6			
7			

(A+B)*C [910]

- Author: Maram Mohamed
- Description: Implementation of (A*B)+C
- GitHub repository
- Wokwi project
- Mux address: 910
- Extra docs
- Clock: 0 Hz

How it works

This project works by designing an AND gate for A and B and an OR gate C. I designed a circuit diagram after writing the expression for F=(A*B)+C.

How to test

I tested to see whether if the output is 1 or 0 by noticing whether if the switch turns on or off. If the output is 1, then the switch turns on and if the output is 0, the switch turns off.

External hardware

The hardware that I used for this project is the Wokwi Template. I also use tiny tapeout to design a circuit for the logic expression that is given to me.

Pinout

#	Input	Output	Bidirectional
0	A	F	
1	B		
2	C		
3			
4			
5			
6			
7			

#	Input	Output	Bidirectional

Example AND Gate [911]

- Author: Amar Mehmedovic
- Description: Example AND Gate
- GitHub repository
- Wokwi project
- Mux address: 911
- Extra docs
- Clock: 0 Hz

How it works

A single AND Gate.

How to test

A single LED can be connected to the output.

External hardware

A single LED.

Pinout

#	Input	Output	Bidirectional
0	A	A AND B	
1	B		
2			
3			
4			
5			
6			
7			

FSM Security Chip [960]

- Author: William Fang
- Description: A 4 states finite state machine security chip.
- GitHub repository
- HDL project
- Mux address: 960
- Extra docs
- Clock: 0 Hz

How it works

This chip has a 4 stage finite state machine to representing the arming, detecting, and triggering of an alarm.

How to test

Have the sensor detect something and see if the alarm is triggered.

External hardware

You will need an alarm and sensor to detect objects.

Pinout

#	Input	Output	Bidirectional
0	Chip On/Off	state	
1	Sensor	state	
2	Alarm On/Off	next state	
3		next state	
4		alarm status	
5			
6			
7			

2 OR Gates and 1 AND Gate [961]

- Author: Kevin Garcia
- Description: 2 OR Gate and 1 AND Gate @Chip Design Workshop
- GitHub repository
- Wokwi project
- Mux address: 961
- Extra docs
- Clock: 0 Hz

How it works

2 OR Gates and 1 AND Gate

How to test

Turning on the LED using OR and AND Gates by switching the switches given

External hardware

1 LED

Pinout

#	Input	Output	Bidirectional
0	A	(A OR C) AND (B OR A)	
1	B		
2	C		
3			
4			
5			
6			
7			

NSF HVCC 25 AND GATE [962]

- Author: Michael Seyffer
- Description: A single AND GATE
- GitHub repository
- Wokwi project
- Mux address: 962
- Extra docs
- Clock: 0 Hz

How it works

This is an AND Gate

How to test

Try all combinations of two bits

External hardware

A single LED

Pinout

#	Input	Output	Bidirectional
0	A	A AND B	
1	B		
2			
3			
4			
5			
6			
7			

Example AND Gate [963]

- Author: Crystal Evans
- Description: Example AND Gate
- GitHub repository
- Wokwi project
- Mux address: 963
- Extra docs
- Clock: 0 Hz

How it works

A single AND Gate.

How to test

A single LED can be connected to the output.

External hardware

A single LED

Pinout

#	Input	Output	Bidirectional
0	A	A	
1	B	B	
2			
3			
4			
5			
6			
7			

MIPS32 Control Unit [964]

- Author: Eehit Mukherjee
- Description: Takes in the last 6 bits and the first 6 bits of a 32-bit instruction to instil control in a MIPS32 CPU
- GitHub repository
- HDL project
- Mux address: 964
- Extra docs
- Clock: 0 Hz

How it works

This project takes in the last 6-bits of a 32-bit wide instruction as the Opcode. The code passed in is from a 32-bit instruction from MIPS. The opcode is then passed through control logic which assigns the control signals to other components in a CPU. The signals are Register Destination, Branch, Memory Read, Memory to Register, ALU Opcode, Memory Write, ALU Source, and Register Write. The ALU OPcode is fed into the ALU control unit which also takes in as input the function bits (first 6-bits of 32-bit instruction) but only the first 4 bits. Then the ALU control Unit outputs the 4 operational code instructions to be fed into an ALU for a CPU. Bits 4 and 5 of the funct. input are set as 0 and 1 , respectively, as default given this chip is designed to work for a limited number of MIPS32 instrucions.

How to test

The project can be used to test certain instructions be R-type instructions (add, subtract, AND, OR, Set on Less Then), Store Word, Load Word, and Branch instructions. You would input your MIPS into a CPU as this chip will be connected with other components in the CPU such as the Program Counter, the ALU, the registers, and the Instruction Memory Unit (Instruction Register). You can probe the physical chip to see if the outputs match the functional code and ALU Operational code for those specfic instructions.

External hardware

Decoder to separate the bits for the code inputs. Registers for writing, reading, inputting MIPS32 instructions.

Pinout

#	Input	Output	Bidirectional
0	OPcode 0	Branch	F0
1	OPcode 1	MemWrite	F1
2	OPcode 2	MemRead	F2
3	OPcode 3	RegWrite	F3
4	OPcode 4	MemtoReg	Op0
5	OPcode 5	ALUSrc	Op1
6		RegDst	Op3
7			Op4

Example AND Gate [965]

- Author: Will
- Description: Example AND Gate
- GitHub repository
- Wokwi project
- Mux address: 965
- Extra docs
- Clock: 0 Hz

How it works

AND gate with LED

How to test

Swtich 1 and 2 will actative the LED depending on the inputs

External hardware

LED

Pinout

#	Input	Output	Bidirectional
0	A	A AND B	
1	B		
2			
3			
4			
5			
6			
7			

Combination Lock [966]

- Author: John Shaughnessy
- Description: It's a combination lock, you can change the combo too
- GitHub repository
- HDL project
- Mux address: 966
- Extra docs
- Clock: 1000 Hz

How it works

It works by being a lock

How to test

Test it

External hardware

Buttons and a singular LED

Pinout

#	Input	Output	Bidirectional
0	SET_PASSWORD	OPEN	TEST PASSWORD
1	B	B	OP1
2	C	C	OP2
3	D	D	OP3
4	E	E	OP4
5	F	F	OP5
6	G	G	OP6
7	H	H	OP7

OR Gates [967]

- Author: Aman Grullon
- Description: OR
- GitHub repository
- Wokwi project
- Mux address: 967
- Extra docs
- Clock: 0 Hz

How it works

Choose either switch 1 OR 2 to turn on LED

How to test

Switch 1 OR 2 would tunr on the LED

External hardware

LED

Pinout

#	Input	Output	Bidirectional
0	A		
1	B	H	
2	C	I	
3	D	J	
4	E		
5	F		
6			
7			

Integer Sequence Term Generator [968]

- Author: Curran Flanders
- Description: Generates last 8 bits of each term from any of eight sequences
- GitHub repository
- HDL project
- Mux address: 968
- Extra docs
- Clock: 100000 Hz

How it works

This project allows the user to select between eight possible modular integer sequences. The available options are the Sylvester, Padovan, Pell, Lucas, and Fibbonacci sequences as well as the perfect squares, the powers of three, and the triangular numbers. The user selects one of these sequences at a time using ui_in[2:0]. The user also selects a clock divider setting correspoding to the desired speed that they want the IC to generate terms of the sequence. The speed varies from 1 term per second to 50,000 terms per second and is set using ui_in[6:3]. Toggle reset to high and back to low to restart the current sequence at its 1st term, and switch enable to low to pause the generator at its current term until enable is set back to high.

How to test

If you do not have access to a logic analyzer or other equipment capable of measuring output pin signals that change frequently, keep the clock divider pins ui_in[6:3] set to a value near the maximum so that the output will be at a sufficiently low frequency to be measured. For a simple test, use ui_in[6:3] to select a term of the sequence and check to see whether the first several terms generated by the IC match the mathematically correct values of the sequence terms. Continue this process for larger terms and different sequences until you are satisfied that their values are correct. Press the reset button after testing each sequence and ensure that the IC restarts at the first term. Use the enable button to pause and restart the chip several times during the test.

Of course, more thorough tests than the one described can be designed for this device. Feel free to check out my test bench in GitHub.

External hardware

All that is needed is controls and displays to set the inputs and read the outputs.

Pinout

#	Input	Output	Bidirectional
0	sequence selector bit 0	current term of selected sequence	
1	sequence selector bit 1	current term of selected sequence	
2	sequence selector bit 2	current term of selected sequence	
3	clock divider setter bit 0	current term of selected sequence	
4	clock divider setter bit 1	current term of selected sequence	
5	clock divider setter bit 2	current term of selected sequence	
6	clock divider setter bit 3	current term of selected sequence	
7		current term of selected sequence	

2-bit to LED output [969]

- Author: Asa
- Description: Converts a 2-bit input to an LED output
- GitHub repository
- Wokwi project
- Mux address: 969
- Extra docs
- Clock: 0 Hz

How it works

Uses AND and NOT gates to select which output is chosen

How to test

Use switch 1 and 2 to change what LED is lit

External hardware

4 LED's

Pinout

#	Input	Output	Bidirectional
0	A	A	
1	B	B	
2		C	
3		D	
4			
5			
6			
7			

One-bit comparator [970]

- Author: Maram Mohamed
- Description: Compares one bit to one bit
- GitHub repository
- Wokwi project
- Mux address: 970
- Extra docs
- Clock: 0 Hz

How it works

My project implements three logic expressions to compare two bits

How to test

Step through the truth table

External hardware

One LED for each of the three outputs

Pinout

#	Input	Output	Bidirectional
0	A	A<B	
1	B	A=B	
2		A>B	
3			
4			
5			
6			
7			

JUST LED's [971]

- Author: MED TOT
- Description: Using AND GATE to turn on an LED's
- GitHub repository
- Wokwi project
- Mux address: 971
- Extra docs
- Clock: 0 Hz

How it works

a few AND Gates and a signle NOT Gate

How to test

4 outputs to LED's and White will always be on unless all other lights are on.

External hardware

4 LEDS

Pinout

#	Input	Output	Bidirectional
0	B1		
1		White	
2	B2	Blue	
3		Purple	
4			
5	R1		
6			
7	R2	Red	

Hand Gesture Logic [973]

- Author: RobBohl
- Description: Just the logic ic for a hand gesture controller
- GitHub repository
- Wokwi project
- Mux address: 973
- Extra docs
- Clock: 0 Hz

How it works

Takes inputs from hand gestures and activates outputs

How to test

Attach a DIP switch to the input pins and LEDs to the outputs to test the logic

External hardware

5 Flex Sensors 5 Op Amp circuits multiple outputs

Pinout

#	Input	Output	Bidirectional
0	A	1	
1	B	2	
2	C	3	
3	D	4	
4	E	5	
5		6	
6			
7			

That-ALU [975]

- Author: Ernesto J Marks
- Description: alu for 4bit binary numbers
- GitHub repository
- Wokwi project
- Mux address: 975
- Extra docs
- Clock: 0 Hz

How it works

This alu was designed to do addition subtration bitwise and and bitwise or based on the two four bit binary numbers input by the user. It does all of these at the same time and the output is selected using a multiplexer. The inputs are grouped in bitwise pairs a with b c with d ect. The resulting numbers are “geca” and “hfdb” where a and b are the least signifigant bits in subtraction “hfdb” is subtracted from “geca”. The two selector bits s1 and s2 decide the function. (00=bwand, 10=bwor, 01=addition, 11=subtraction) Explain how your project works

How to test

testing can be conducted by selecting an operation picing two input numbers and comparing the output to hand by calcultions Explain how to use your project

External hardware

Connect the 5 outputs to leds to see the resultant number List external hardware used in your project (e.g. PMOD, LED display, etc), if any

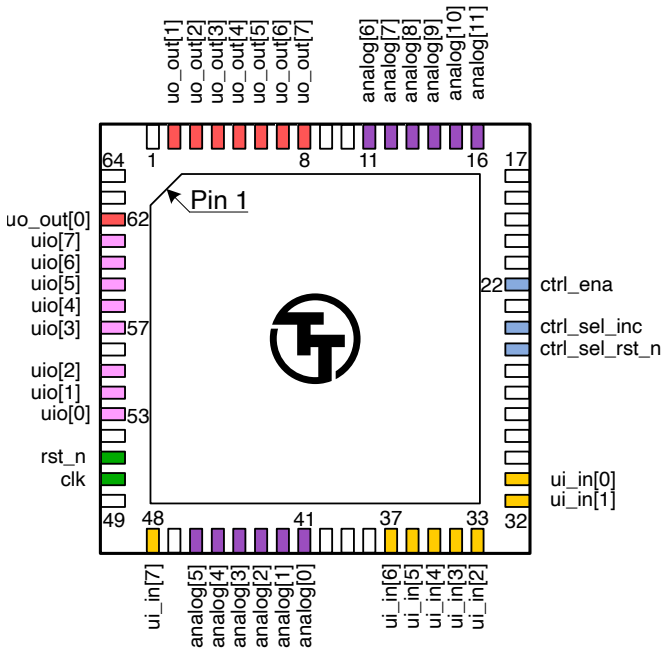
Pinout

#	Input	Output	Bidirectional
0	a	i	s1
1	b	j	s2

#	Input	Output	Bidirectional
2	c	k	
3	d	l	
4	e	m	
5	f		
6	g		
7	h		

Pinout

The chip is packaged in a 64-pin QFN package. The pinout is shown below.



Bottom View

Note: you will receive the chip mounted on a breakout board. The pinout is provided for advanced users, as most users will not need to solder the chip directly.

The Tiny Tapeout Multiplexer

Overview

The Tiny Tapeout Multiplexer distributes a single set of user IOs to multiple user designs. It is the backbone of the Tiny Tapeout chip.

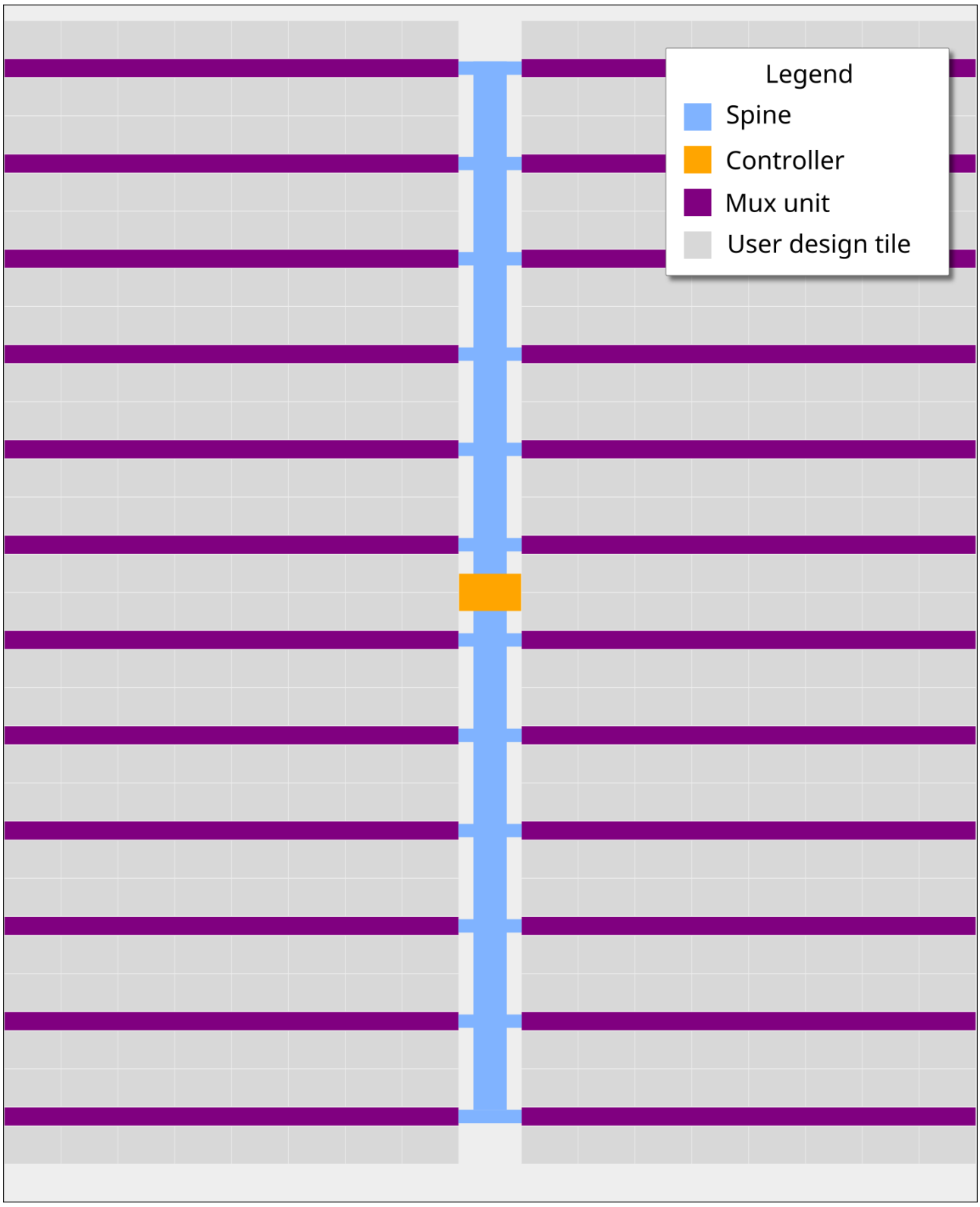
It has the following features:

- 10 dedicated inputs
- 8 dedicated outputs
- 8 bidirectional IOs
- Supports up to 512 user designs (32 mux units, each with up to 16 designs)
- Designs can have different sizes. The basic unit is a called a tile, and each design can occupy up to 16 tiles.

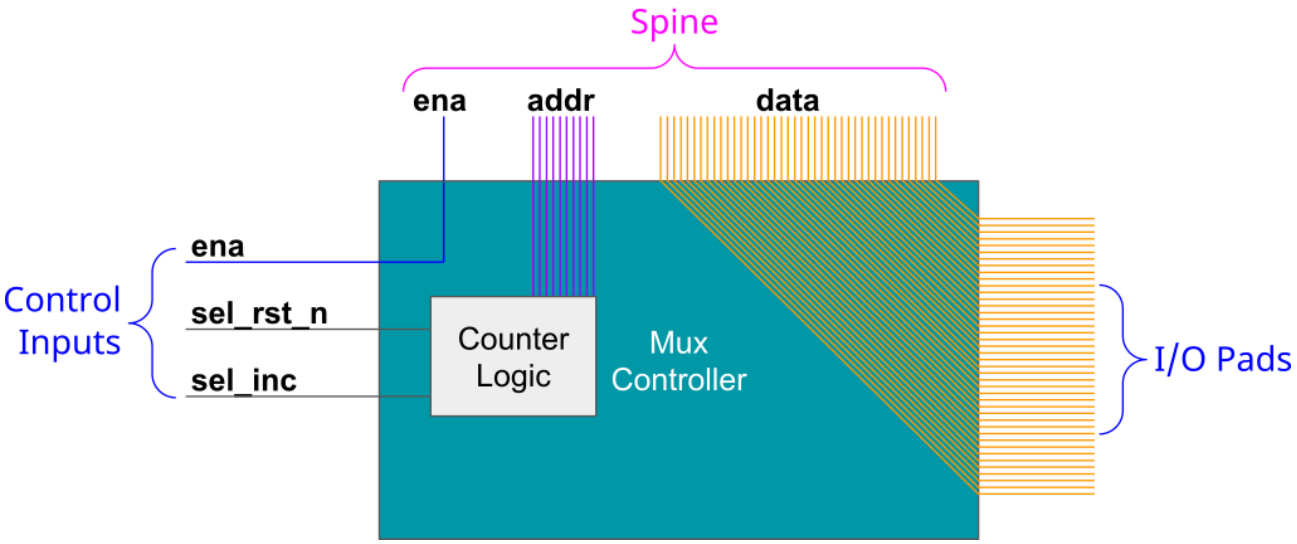
Operation

The multiplexer consists of three main units:

1. The controller - used to set the address of the active design
2. The spine - a bus that connects the controller with all the mux units
3. Mux units - connect the spine to individual user designs


Legend
Spine
Controller
Mux unit
User design tile

The Controller

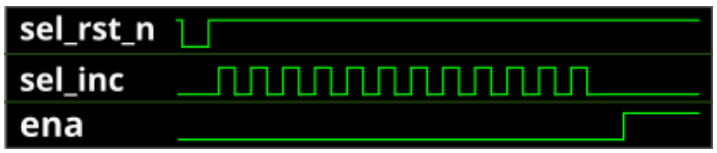


The mux controller has 3 inputs lines:

Input	Description
`ena`	Sent as-is (buffered) to the downstream mux units
`sel_rst_n`	Resets the internal address counter to 0 (active low)
`sel_inc`	Increments the internal address counter by 1

It outputs the address of the currently selected design on the `si_sel` port of the spine (see below).

For instance, to select the design at address 12, you need to pulse `sel_rst_n` low, and then pulse `sel_inc` 12 times:



Internally, the controller is just a chain of 10 D flip-flops. The `sel_inc` signal is connected to the clock of the first flip-flop, and the output of each flip-flop is connected to the clock of the next flip-flop. The `sel_rst_n` signal is connected to the reset of all flip-flops.

The following Wokwi projects demonstrates this setup: https://wokwi.com/projects/364347807664031745. It contains an Arduino Nano that decodes the currently selected mux address and displays it on a 7-segment display. Click on the button labeled `RST_N` to reset the counter, and click on the button labeled `INC` to increment the counter.

The Spine

The controller and all the muxes are connected together through the spine. The spine has the following signals going on it:

From controller to mux:

- `si_ena` - the `ena` input
- `si_sel` - selected design address (10 bits)
- `ui_in` - user clock, user `rst_n`, user inputs (10 bits)
- `uio_in` - bidirectional I/O inputs (8 bits)

From mux to controller:

- `uo_out` - User outputs (8 bits)
- `uio_oe` - Bidirectional I/O output enable (8 bits)
- `uio_out` - Bidirectional I/O outputs (8 bits)

The only signal which is actually generated by the controller is `si_sel` (using `sel_rst_n` and `sel_inc`, as explained above). The other signals are just going through from/to the chip IO pads.

The Multiplexer (The Mux)

Each mux branch is connected to up to 16 designs. It also has 5 bits of hard-coded address (each unit gets assigned a different address, based on its position on the die).

The mux implements the following logic:

If `si_ena` is 1, and `si_sel` matches the mux address, we know the mux is active. Then, it activates the specific user design port that matches the remaining bits of `si_sel`.

For the active design:

- `clk`, `rst_n`, `ui_in`, `uio_in` are connected to the respective pins coming from the spine (through a buffer)
- `uo_out`, `uio_oe`, `uio_out` are connected to the respective pins going out to the spine (through a tristate buffer)

For all others, inactive designs (including all designs in inactive muxes):

- `clk`, `rst_n`, `ui_in`, `uio_in` are all tied to zero

- `uo_out`, `uio_oe`, `uio_out` are disconnected from the spine (the tristate buffer output enable is disabled)

Pinout

mprj_io pin	Function	Signal	QFN64 pin
0	Input	ui_in[0]	31
1	Input	ui_in[1]	32
2	Input	ui_in[2]	33
3	Input	ui_in[3]	34
4	Input	ui_in[4]	35
5	Input	ui_in[5]	36
6	Input	ui_in[6]	37
7	Analog	analog[0]	41
8	Analog	analog[1]	42
9	Analog	analog[2]	43
10	Analog	analog[3]	44
11	Analog	analog[4]	45
12	Analog	analog[5]	46
13	Input	ui_in[7]	48
14	Input	clk †	50
15	Input	rst_n †	51
16	Bidirectional	uio[0]	53
17	Bidirectional	uio[1]	54
18	Bidirectional	uio[2]	55
19	Bidirectional	uio[3]	57
20	Bidirectional	uio[4]	58
21	Bidirectional	uio[5]	59
22	Bidirectional	uio[6]	60
23	Bidirectional	uio[7]	61
24	Output	uo_out[0]	62
25	Output	uo_out[1]	2
26	Output	uo_out[2]	3
27	Output	uo_out[3]	4
28	Output	uo_out[4]	5
29	Output	uo_out[5]	6
30	Output	uo_out[6]	7
31	Output	uo_out[7]	8
32	Analog	analog[6]	11
33	Analog	analog[7]	12

mprj_io pin	Function	Signal	QFN64 pin
34	Analog	analog[8]	13
35	Analog	analog[9]	14
36	Analog	analog[10]	15
37	Analog	analog[11]	16
38	Mux Control	ctrl_ena	22
39	Mux Control	ctrl_sel_inc	24
40	Mux Control	ctrl_sel_rst_n	25
41	Reserved	(none)	26
42	Reserved	(none)	27
43	Reserved	(none)	28

† Internally, there's no difference between `clk`, `rst_n`, and `ui_in` pins. They are all just bits in the `pad_ui_in` bus. However, we use different names to make it easier to understand the purpose of each signal.

Funding

IHP PDK support for Tiny Tapeout was funded by The SwissChips Initiative.

The manufacturing of Tiny Tapeout IHP 0p2 silicon was funded by the German BMBF project FMD-QNC (16ME0831).

Team

Tiny Tapeout would not be possible without a lot of people helping. We would especially like to thank:

- Uri Shaked for wokwi development and lots more
- Patrick Deegan for PCBs, software, documentation and lots more
- Sylvain Munaut for help with scan chain improvements
- Mike Thompson and Mitch Bailey for verification expertise
- Tim Edwards and Harald Pretl for ASIC expertise
- Jix for formal verification support
- Proppy for help with GitHub actions
- Maximo Balestrini for all the amazing renders and the interactive GDS viewer
- James Rosenthal for coming up with digital design examples
- All the people who took part in Tiny Tapeout 01 and volunteered time to improve docs and test the flow
- The team at YosysHQ and all the other open source EDA tool makers
- Jeff and the Efabless Team for running the shuttles and providing Open-Lane and sponsorship
- Tim Ansell and Google for supporting the open source silicon movement
- Zero to ASIC course community for all your support
- Jeremy Birch for help with STA